DOTE 6751 · DATABASE SYSTEMS

Reverse-Engineering LinkedIn: A Relational Model of the Hiring Loop

DOTE 6751 Database Systems · Group Project · November 2026
Xuanmo AN · Yi HOU · Zhao Han HUANG · Jinyu LI · Yitong NI · Shujing YE · Shiyao ZHANG

1. Introduction and Scope

A professional network presents itself as a set of screens: a profile, a job card, a status badge, an applicant list. Each screen is the visible edge of a query, and each query presupposes a schema. This project reconstructs that schema for one closed process — the hiring loop that begins with an advertised vacancy and ends with a hire — and demonstrates that the reconstruction is sufficient to answer the questions the product itself answers. The method proceeds in three stages: observe the states and transitions the product exposes, derive the entity and relationship sets those states presuppose, and test the derivation by populating it and asking the questions again. The work is deliberately not an attempt to reproduce a commercial physical design, which is distributed, partially denormalised and unavailable. It is an exercise in logical reverse engineering: deriving the smallest normalised model that explains the observed behaviour, and then testing that model against a populated instance.

Three considerations fix the boundary of the study. First, the loop has a terminal event with a timestamp, which makes cycle time and conversion measurable rather than rhetorical. Second, it necessarily involves two parties and a recommendation between them, so the social graph is represented by data rather than asserted in prose. Third, it forces the model to distinguish what a person knows from what a vacancy demands, which is the condition for any defensible statement about a skill gap. The resulting scope is twelve relations, seventeen foreign keys, twenty-five check constraints and four triggers, populated with three hundred members, sixty employers, one hundred and twenty postings and two hundred and fifty applications. Eighteen queries exercise the model in three escalating groups: reporting, business analysis, and advanced analytics using common table expressions, window functions and recursion.

2. The Hiring Loop as a Business Process

The process admits a linear description with a branching exit. An employer creates a posting and declares the skills it requires, distinguishing hard requirements from preferences and stating a minimum proficiency for each. A member applies through one of three channels — direct, referral, or recruiter sourcing — and the channel is recorded on the application rather than inferred later. A recruiter screens the application; successful candidates are interviewed in numbered rounds, each with an interviewer, a format, a schedule and an outcome; the process terminates in exactly one of three states: hired, rejected, or withdrawn by the candidate.

Two properties of this process determine the shape of the schema. The first is that the application is not a passive link between a person and a vacancy but an entity with its own lifetime, its own attributes and its own dependants. Interviews belong to it, a referral is attached to it, and its status changes over weeks. The second is that the process is governed by rules that no single row can enforce: an application may not skip from submission to hire, an interview may not precede the application it belongs to, a referral may only be created by someone currently employed by the hiring company, and a posting may not hire more people than it declared openings for. These are exactly the constraints that a relational design must either express declaratively or explicitly decline to express, and the project's constraint inventory records which choice was made in each case and why.

3. Conceptual Design

The conceptual model is expressed in Chen notation and contains twelve entity sets and fifteen relationship sets. Four entity sets describe the person — member, education, experience and connection — and three describe demand: company, job_posting and skill. Two associative entities, member_skill and job_skill, carry the attributes that make matching meaningful: proficiency and endorsements on one side, importance and a minimum level on the other. The remaining three — application, interview and referral — constitute the loop itself. Cardinalities follow the process: a company advertises many postings, a posting receives many applications, an application schedules one or more interviews, and a referral stands in a one-to-one relation to the application it produced.

Three decisions in this layer deserve comment. The current employer is not modelled as an attribute of member, because it is already implied by the experience relation once one row is marked current; storing it twice would create a redundancy that the database management system could not police. The referral relation stores only the referrer and the bonus, because the candidate and the posting are functions of the application to which it is attached. The connection relation is an undirected edge stored once in canonical order, a choice that halves storage and turns the question of whether two members are connected into a single primary-key lookup, at the cost of an explicit direction flip in the recursive query that later traverses it.

4. Relational Design and Normalisation

The reduction to relations follows the mapping rules of Silberschatz, Korth and Sudarshan, Database System Concepts, seventh edition, section 6.7. Strong entity sets are taken over directly with their identifiers as primary keys. One-to-many relationship sets migrate the key of the one side into the many side, which accounts for twelve of the seventeen foreign keys. The one-to-one relationship between application and referral places the application identifier in the referral relation and constrains it to be unique, since the referral side is the one with total participation. The many-to-many relations, including the self-referencing connection, become relations of their own keyed by the union of the participating identifiers.

Normalisation is proved table by table rather than asserted. For each relation the functional dependencies are enumerated, candidate keys are identified from the primary key and any enforced unique constraint, and the third-normal-form test of section 7.3.2 is applied: for every non-trivial dependency, the determinant must be a superkey or the dependent attribute must be prime. No relation required decomposition. The stricter Boyce-Codd test also holds, because every determinant identified is a candidate key; this is reported as a stronger result rather than as the design target. Two judgement calls are recorded because a mechanical reading of the data would get them wrong: a recruiter's identifier does not determine the hiring company, since a recruiter may change employer while historical postings remain with the company that commissioned them; and the headline attribute is free text that paraphrases the current position rather than a projection of it, so it is treated as a snapshot and not as a derived column.

5. Implementation, Data and Integrity

The schema is implemented in MySQL 8.0 with the InnoDB engine, the utf8mb4 character set and the utf8mb4_0900_ai_ci collation. Integrity is expressed at four levels. Primary keys, foreign keys, unique constraints and check constraints cover the rules that a single row or a single reference can decide. Four triggers cover the rules that cannot: the application state machine and its decision timestamp, the eligibility and contiguity of interview rounds, and the validity of a referral against the employment record of the referrer. Referential actions are chosen deliberately: content that belongs to an entity is deleted with it, whereas records that attribute a decision to a person are protected by restrict, because deleting them would destroy evidence.

The instance is generated deterministically from a fixed random seed and is designed to satisfy the process rules rather than merely to load. A hired application has at least three passed interviews; a rejection that followed an interview ends in a failed round; a terminal status always carries a decision timestamp; every referral application has a referrer inside the hiring company; no posting hires beyond its openings; and no member applies to their current employer. Twenty-two cross-table checks verify these invariants after every rebuild, and fourteen deliberately invalid statements — a duplicated application, an inverted salary band, a second current position, an illegal status jump, a referral from outside the company — are each refused by the expected object with the expected error code.

6. Analytical Findings and Limitations

The eighteen queries are organised in three groups. The first six report on the current state: open roles in a market with their pay bands and age, the most contested postings, the skills the market demands, and a company scorecard. The next seven address business questions that require several relations at once: conversion by sourcing channel, the return on the referral programme, individual skill gaps, cycle time by seniority, interviewer calibration, internal skill shortages and stalled screening stages. The final five use common table expressions, window functions and a recursive traversal of the connection graph.

Three findings illustrate what the model makes visible. Referral applications convert to hires at three times the rate of direct applications in this instance, and the difference arises after the interview rather than at screening, which is a statement no single table could support. Interview rounds account for fifteen to twenty days of a thirty-day hire, so the binding constraint is scheduling latency rather than deliberation. And a two-hop traversal of the social graph, intersected with a posting's hard requirements, produces a short list of warm-introduction candidates that the product itself would charge to assemble. The limitations are stated with equal precision: the instance is synthetic, one currency is used, the schema keeps a single decision timestamp rather than a transition log, the education natural key is not enforced, and the graph traversal is bounded at two hops. Each limitation is a design decision recorded in the rationale, and each has a concrete remedy that a production system would adopt.

The deliverable is therefore reproducible rather than merely reported. The generator, the schema scripts, the query harness and the verification suite are all part of the submission; a reader who rebuilds the database obtains the same row counts, the same invariant checks and the same result files, and can contest any number in this narrative by rerunning the query that produced it.